

Module 04 | Learning in Perceptrons – From Gradient Descent to Loss Functions

My Solution to Practice Problems

Table of Contents

- Q01. Why does the perceptron algorithm use class labels -1 and $+1$ instead of 0 and 1 ? Explain how this choice affects the perceptron update rule.
- Q02. Gradient descent usually updates weights by subtracting the gradient. Why does the perceptron update rule add the term ηyx instead?
- Q03. Explain in your own words how the perceptron learning rule can be seen as a form of gradient descent. Also mention one limitation of the perceptron loss function.

Q01. Why does the perceptron algorithm use class labels -1 and $+1$ instead of 0 and 1 ? Explain how this choice affects the perceptron update rule.

The question refers to two choices below about Perceptron's Activation Function, both with Weighted Sum, $z \equiv wx + b$.

$$a_{0,1}(z) \equiv \begin{cases} 1, & \text{if } z \geq 0 \\ 0, & \text{if } z < 0 \end{cases}$$

$$a_{\pm 1}(z) \equiv \begin{cases} 1, & \text{if } z \geq 0 \\ -1, & \text{if } z < 0 \end{cases}$$

For $a_{\pm 1}(z)$

Standard Loss Function is

$$L_{W,b}(x) \equiv \max(0, -yz)$$

In case of Correct Classification, y and z do not disagree in sign $\implies yz \geq 0 \implies -yz \leq 0 \implies L \equiv 0$.

In case of Misclassification, y and z disagree in sign $\implies yz < 0 \implies -yz > 0 \implies L \equiv -yz \equiv -y(wx + b)$.

Then, according to the Gradient Descent Optimization, the Perceptron Learning Rule is

$$\begin{aligned} w_{\text{new}} &= w_{\text{old}} - \eta \frac{\partial L}{\partial w} \\ &= \begin{cases} w_{\text{old}} - \eta \frac{\partial}{\partial w}(0), & \text{for Correct Classification} \\ w_{\text{old}} - \eta \frac{\partial}{\partial w}(-y(wx + b)), & \text{for Misclassification} \end{cases} \\ &= \begin{cases} w_{\text{old}}, & \text{for Correct Classification} \\ w_{\text{old}} + \eta yx, & \text{for Misclassification} \end{cases} \end{aligned}$$

$$\begin{aligned} b_{\text{new}} &= b_{\text{old}} - \eta \frac{\partial L}{\partial b} \\ &= \begin{cases} b_{\text{old}} - \eta \frac{\partial}{\partial b}(0), & \text{for Correct Classification} \\ b_{\text{old}} - \eta \frac{\partial}{\partial b}(-y(wx + b)), & \text{for Misclassification} \end{cases} \\ &= \begin{cases} b_{\text{old}}, & \text{for Correct Classification} \\ b_{\text{old}} + \eta y, & \text{for Misclassification} \end{cases} \end{aligned}$$

For $a_{0,1}(z)$

A standard Loss Function is

$$L_{W,b}(x) \equiv (y - \hat{y})z$$

In case of Correct Classification, $(y - \hat{y}) = 0 \implies L \equiv 0$. In this case, either Gradient Ascent or Gradient Descent is fine.

In case of Misclassification, $(y - \hat{y}) \neq 0$ and $L \equiv (y - \hat{y}) z \equiv (y - \hat{y}) (wx + b)$.

For False Positive Misclassification, $y - \hat{y} = 0 - 1 = -1 \implies L < 0$. This case need Gradient Ascent.

For False Negative Misclassification, $y - \hat{y} = 1 - 0 = 1 \implies L < 0$. This case too need Gradient Ascent.

Then, according the Gradient Ascent Optimization, the Perceptron Learning Rule is

$$\begin{aligned} w_{\text{new}} &= w_{\text{old}} + \eta \frac{\partial L}{\partial w} \\ &= \begin{cases} w_{\text{old}} + \eta \frac{\partial}{\partial w} (0), & \text{for Correct Classification} \\ w_{\text{old}} + \eta \frac{\partial}{\partial w} ((y - \hat{y}) (wx + b)), & \text{for Misclassification} \end{cases} \\ &= \begin{cases} w_{\text{old}}, & \text{for Correct Classification} \\ w_{\text{old}} + \eta (y - \hat{y}) x, & \text{for Misclassification} \end{cases} \end{aligned}$$

$$\begin{aligned} b_{\text{new}} &= b_{\text{old}} + \eta \frac{\partial L}{\partial b} \\ &= \begin{cases} b_{\text{old}} + \eta \frac{\partial}{\partial b} (0), & \text{for Correct Classification} \\ b_{\text{old}} + \eta \frac{\partial}{\partial b} ((y - \hat{y}) (wx + b)), & \text{for Misclassification} \end{cases} \\ &= \begin{cases} b_{\text{old}}, & \text{for Correct Classification} \\ b_{\text{old}} + \eta (y - \hat{y}), & \text{for Misclassification} \end{cases} \end{aligned}$$

Comparison

Perceptron	Activation Function	
	$a_{0,1}(z)$	$a_{\pm 1}(z)$
Loss Function	$L_{W,b}(x) \equiv (y - \hat{y}) z$	$L_{W,b}(x) \equiv \max(0, -yz)$
Learning Rule	$w_{\text{new}} = w_{\text{old}} + \eta (y - \hat{y}) x$	$w_{\text{new}} = w_{\text{old}} + \eta y x$

$$b_{\text{new}} = b_{\text{old}} + \eta(y - \hat{y})$$

$$b_{\text{new}} = b_{\text{old}} + \eta y$$

So, that's how our choice of class labels for Perceptron affects both Loss Function and Learning Rule.

Q02. Gradient descent usually updates weights by subtracting the gradient. Why does the perceptron update rule add the term $\eta y x$ instead?

Yes, by subtracting the gradient of the **Loss Function**.

In the special case of Perceptron, with $a_{\pm 1}(z)$ as the activation function, a standard loss function is

$$L_{W,b}(x) \equiv \max(0, -yz).$$

Its gradients are

$$\begin{aligned} \frac{\partial L}{\partial w} &= -yx \\ \frac{\partial L}{\partial b} &= -y \end{aligned}$$

That's why the Perceptron, using Gradient Descent Optimization in its own special case, updates its weights and bias with a term with y and x as factors, and also η , as generally suggested by Gradient Descent.

Q03. Explain in your own words how the perceptron learning rule can be seen as a form of gradient descent. Also mention

one limitation of the perceptron loss function.

Perceptron Learning Rule, is not an equivalent form of Gradient Descent, but an special form of Gradient Descent. Perceptron Learning Rule is derived applying the General Gradient Descent Optimization Algorithm on the special case of perceptron. See [the Derivation](#).